

熵不对称与评测偏差：SEAC 算法在 RWARE 仓储环境中的训练、分析与改进

田照亿¹

1. 引言

基于 RWARE 仓储环境，我自行编写了训练评估框架，实现了 IAC、SNAC、SEAC (Christianos et al., 2021) 三个基线算法，并在 tiny-2ag 和 small-4ag 两个规模下进行了训练与评估。完整代码已开源。¹

通过对比训练曲线，我发现 SEAC 算法的训练 reward 显著高于 IAC，但这并非源于更好的策略质量。进一步分析表明，SEAC 的跨智能体损失项中不含熵正则化，导致其策略天然低熵——训练时的随机采样几乎等同于确定性推理，从而推高了 episodic reward。相比之下，IAC 因较高的探索熵，训练得分被随机动作拉低。这一现象提示，当不同算法的策略熵存在系统性差异时，训练曲线可能无法直接反映策略的真实水平。为此，我设计了 SEAC-Pooled 算法在跨智能体经验上引入加权熵以消除不对称，并通过温度扫描消融实验在不同熵水平上对齐各算法进行公平对比。结果表明，参数不共享的算法 (IAC、SEAC、SEAC-Pooled) 在统一评估下的渐近性能接近，差异主要体现在收敛速率。

在此基础上，我进一步探索了 SEAC-SIL 算法改进方向，通过回放高优势经验来加速稀疏奖励下的收敛。受限于计算资源，所有实验基于单一随机种子。

2. 问题设置与方法

RWARE 模拟了一个多机器人物流仓储场景：若干机器人在网格仓库中移动，需要将货架搬运至指定交付点。该环境的难点主要体现在两方面。一是奖励稀疏——智能体仅在成功交付被请求的货架时获得 +1 奖励 (INDIVIDUAL 模式)，整个搬运链条中的导航、捡取、归还空货架等中间步骤均无任何反馈信号。二是部分可观测——每个智能体仅能感知以自身为中心的 3×3 局部网格，观测维度为 71 (自身坐标、朝向、载货状态、所在区域类型，以及 9 个网格格点的智能体

与货架信息)。

我实现了三个基线算法。IAC (Independent Actor-Critic) 为每个智能体配备独立的策略与价值网络，仅用自身数据更新。SNAC (Shared Network Actor-Critic) 则使所有智能体共享同一个网络参数，梯度跨智能体累积后一次更新。SEAC (Christianos et al., 2021) 在独立网络的基础上引入了跨智能体经验共享：每个智能体在更新时不仅使用自己的 rollout 数据，还通过重要性采样比率 $\pi_i(a|s)/\pi_j^{\text{old}}(a|s)$ 利用其他智能体的经验，从而大幅提升样本效率。

实验在 tiny-2ag (11×10 网格, 2 智能体) 和 small-4ag (20×10 网格, 4 智能体) 两个规模下进行。每个 episode 最多 500 步，环境中同时存在的被请求货架数量与智能体数量相同 (即 2 或 4 个)。所有算法均基于 A2C 框架：采用 5 步 n-step return ($\gamma = 0.99$)，网络为两层 128 维共享主干 MLP，actor 与 critic 各一个线性输出头，正交初始化 (actor 输出层 gain=0.01)，使用 Adam 优化器 ($\text{lr} = 3 \times 10^{-4}$, $\epsilon = 10^{-3}$)。训练时并行 4 个环境、每次 rollout 5 步，共训练 80M (tiny) 和 150M (small) 步。超参数与 SEAC 原论文保持一致以消除调参偏差。

3. 基线训练结果与分析

3.1. 基线复现与收敛分析

图 1(a)(d) 展示了三个基线算法在 tiny-2ag 和 small-4ag 上的训练 episodic reward 曲线。两个环境中均观察到了与 SEAC 原论文一致的趋势：最终收敛水平 $\text{SEAC} > \text{IAC} > \text{SNAC}$ 。在收敛速度方面，两个环境中均观察到与原文一致的倍数关系：SEAC 进入快速增长期所需的训练步数约为 IAC 的 $1/N$ ，其中 N 为智能体数量。例如在 small-4ag ($N=4$) 中，SEAC 约在 5M 步附近开始上升，IAC 则需约 20M 步；tiny-2ag ($N=2$) 中同样呈现约两倍的比例。SEAC 论文将此归因于跨智能体经验共享带来的样本效率提升——每个智能体不再仅依赖自身经验，而是从所有智能体的交互中共同学习。

SNAC 则始终表现最弱。共享同一策略网络使得所有智能体的行动高度相似，难以在有限的仓储空间内形成有差异的行为。这一结果提示，在 RWARE 这类需要智能体独立做决策的任务中，参数共享可能是一种

¹ 北京大学，北京，中国。Correspondence to: 田照亿 <2400013763@pku.edu.cn>.

Proceedings of the 39th International Conference on Machine Learning, Baltimore, Maryland, USA, PMLR 162, 2022. Copyright 2022 by the author(s).

¹<https://github.com/Zhaoyi-Tian/Rware-seac-analysis>

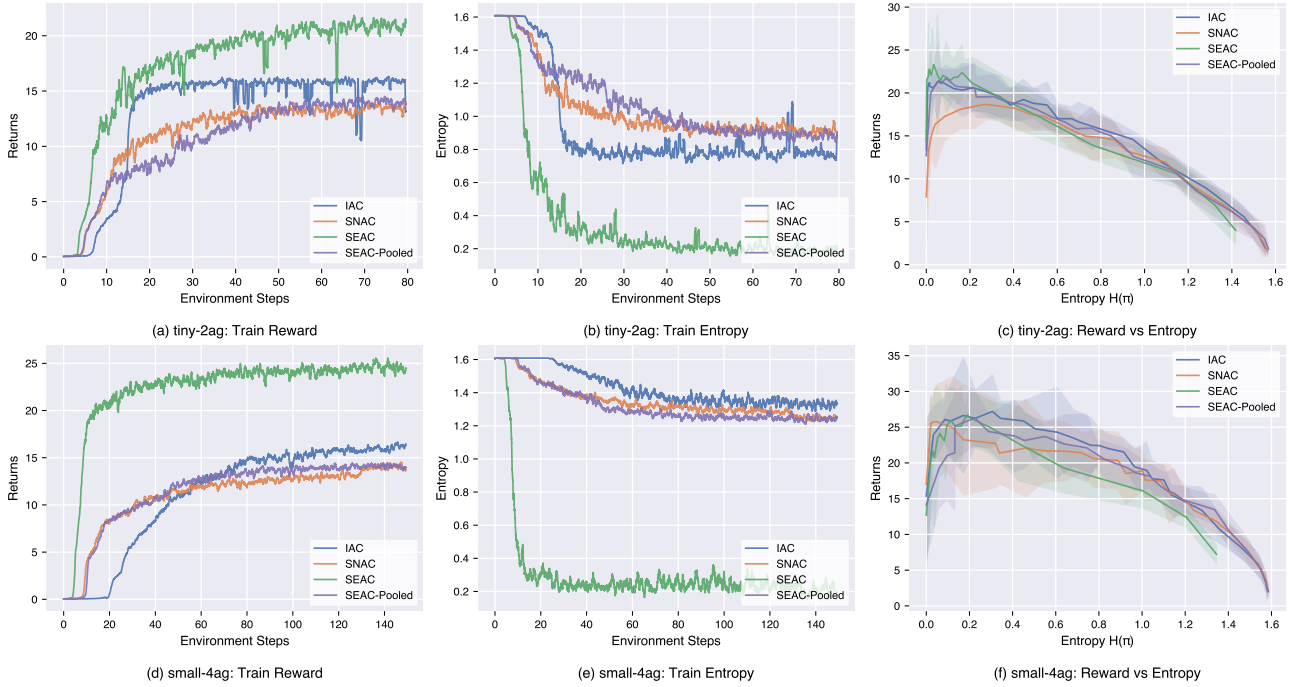


Figure 1. 各算法在 tiny-2ag (上排) 和 small-4ag (下排) 上的训练与评估结果。(a)(d) 训练 episodic reward 曲线, 已做 50 点滑动平均平滑; (b)(e) 训练策略熵曲线, 同样经 50 点滑动平均; (c)(f) 不同温度下评估 reward 与策略熵的关系, 折线经 3 点滑动平均, 横轴为策略经温度 T 缩放后的实际熵 $H(\pi_T)$, 纵轴为 10 个随机种子下的平均 episode reward。

不恰当的归纳偏置。

3.2. 熵不对称与公平比较

尽管训练曲线呈现 $SEAC > IAC > SNAC$ 的排序, 但在确定性评估 ($\arg\max$, 即 $T=0$) 下的表现却有所不同: IAC 和 SNAC 的评估得分相比训练阶段有较明显的提升, 而 SEAC 几乎没有变化。观察图 1(b)(e) 的策略熵曲线可以发现原因。SEAC 的策略熵在训练过程中迅速下降, 最终在两个规模下分别稳定在约 0.19 (tiny) 和 0.22 (small) 的极低水平 (max 概率约 0.94, 几乎为确定性策略); 相比之下, IAC 的策略熵维持在约 0.79 (tiny) 和 1.33 (small), 保持了有意义的探索空间。SEAC-Pooled 修正后熵回升至约 0.89 (tiny) 和 1.24 (small), 与其他算法持平。SEAC 低熵的根源在于其跨智能体损失项中不含熵正则化——每个智能体在使用其他智能体的经验更新时, 只计算策略梯度和价值损失, 不施加任何探索压力。随着跨智能体数据在总损失中占比增大, 整体的熵项被稀释, 策略因而不可逆地趋向确定性。

这种熵的失衡直接导致训练 reward 的对比失去公平性。SEAC 原文报告, 在最困难的 small-4ag 任务中, SEAC 的最终平均回报比 IAC 和 SNAC 分别高出约 70% 和 160% (Christianos et al., 2021)。但这一优势至少部分来自采样方式的差异: 训练时 SEAC 的低熵

使随机采样接近确定性推理, 而 IAC 因高探索熵损失了大量步数。换言之, 训练 reward 曲线混合了策略的真实能力和采样噪声, 无法直接反映算法优劣。

为修正这一偏差, 我设计了 SEAC-Pooled 算法。与原版 SEAC 的核心区别在于, 它为跨智能体经验项补上了重要性加权的熵损失, 使得整个 loss 中熵的占比不再因跨智能体数据的加入而被稀释。从图 1(b)(e) 可以看到, SEAC-Pooled 的策略熵与其他算法保持在同一水平, 有效解决了熵不对称问题。但值得注意的是, 修正后的 SEAC-Pooled 在训练 reward 上低于 IAC (见图 1(a)(d))。尽管受限于单一随机种子, 这一结果的可靠性有限, 但它提示了一个可能性: SEAC 的主要贡献在于通过经验共享提升收敛速度, 其提升最终得分的效果可能并不如训练曲线所暗示的那样显著。

为了在更公平的条件下比较各算法的策略质量, 我将策略分解为两个可分离的维度: 一是策略的“锐度”, 由 softmax 的温度 T 控制, 反映熵的高低; 二是 logits 的相对排序, 反映策略在给定状态下对各动作的偏好结构。通过对已训好的模型施加统一温度进行采样评估, 可以绘制 reward 随策略熵 (经 T 缩放后的熵) 变化的曲线, 从而在同一锐度下比较不同算法的排序质量。图 1(c)(f) 展示了这一分析的结果。

在 tiny-2ag 上, SEAC 在较低熵水平下保持微弱领先, 但在 small-4ag 上, 趋势更加微妙: 当策略趋于确定

性 ($T \rightarrow 0$) 时, IAC 的评估得分反超 SEAC, 此时所有算法的 reward 均偏低——低熵策略在较大的仓储空间中容易陷入死锁, 种子扫描中甚至出现了 SEAC 在个别初始配置下得分为 0 的极端情况。随着温度升高、熵增大, 随机性带来的探索有助于跳出局部最优, reward 逐渐上升; 但当熵过大时, 随机动作的代价又拖累了表现。各算法在各自最优的熵区间内取得的最高 reward 较为接近。

综合以上分析, 可以形成两点认识。第一, SEAC 的核心价值在于通过跨智能体经验共享显著提升了收敛速度 (约为 IAC 的 N 倍), 这一优势是实质性的。第二, 训练 reward 的差异受到熵不对称的严重污染, 在修正这一偏差后, 参数不共享的算法之间并未显现出显著的性能差距。这提示后续算法改进的重点或许不在于仅仅凭借简单算法改进来得到更高的最终得分, 而在于继续提高样本利用率、引入更丰富的信息来源, 或从机制上缓解多智能体之间的碰撞与死循环 (引入适度的不确定性可能是一种简单有效的方案)。

4. SEAC-SIL: 自模仿学习加速收敛

前文的分析表明, 在修正熵不对称后, 各参数不共享算法之间的性能差距并不显著, 差异更多体现在收敛速率上。这引出了一个自然的问题: 能否在 SEAC 跨智能体经验共享的基础上, 进一步从其他维度提高样本利用率? SEAC 的跨智能体共享本质上是利用同一时刻不同智能体的数据——一个空间维度的经验复用。我尝试从时间维度引入另一种经验复用: 将过去 episode 中的高质量经验存储起来, 在后续训练中反复学习。

这一思路直接催生了 SEAC-SIL (Self-Imitation Learning)。其核心设计十分简洁: 在每个智能体的更新过程中, 将优势函数为正 (即实际回报高于当前价值估计) 的 transition 存入一个 FIFO 回放缓冲; 每次更新时, 除了标准的 A2C loss 和 SEAC 跨智能体 loss 外, 额外从缓冲中采样一批数据, 以重要性采样修正后的策略梯度和价值损失进行训练。重要性比率 $\pi_{\text{new}}(a|s)/\pi_{\text{old}}(a|s)$ 保证了即使缓冲中的数据由旧策略采集, 梯度估计在期望上仍是无偏的。SIL loss 与 SEAC 跨智能体 loss 共享完全相同的数学形式 (ratio-weighted policy loss + value loss), 唯一的区别在于数据来源——前者来自过去的自己, 后者来自同一时刻的其他智能体。这一对称设计不仅仅是审美上的简洁: 它意味着两种经验复用在同一套数学框架下运行, 便于分析和扩展。相应的, IAC-SIL 则是将同样的 SIL 机制直接应用于独立 A2C, 作为对照以剥离 SEAC 跨智能体共享与 SIL 的各自贡献。

图 2 展示了四组算法在 small-4ag 上 50M 步的训练曲线。进入快速增长长期的速度排序为 SEAC-SIL > IAC-SIL > SEAC > IAC, 恰好印证了双层经验复用的叠加效果: 跨智能体共享 (SEAC > IAC) 和纵向自模仿 (SIL > 非 SIL) 各自的加速效应可以累计。值得注意的是, IAC-SIL 在收敛初期超越了原版 SEAC, 说明

仅凭回放过去的高质量数据, 样本效率已可与跨智能体共享相媲美; 而 SEAC-SIL 将两者结合后, 取得了最快的增速。

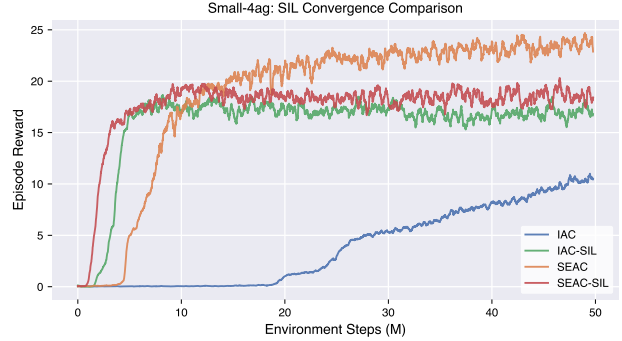


Figure 2. IAC、SEAC 及各自 SIL 变体在 small-4ag 上的训练 episodic reward 曲线 (50M 步, 50 点滑动平均平滑)。SIL 配置: 缓冲区容量 10000, 损失权重 $\lambda_{\text{SIL}} = 1.0$, 每次采样 batch 大小 $5 \times 4 \times (N-1) = 60$, 与 SEAC 跨智能体项对称。

5. 总结与讨论

本报告围绕 SEAC 算法在 RWARE 仓储环境中的表现, 进行了系统的复现、分析与改进探索。实验所使用的 RWARE 环境采用 INDIVIDUAL 奖励模式——仅成功交付货架的智能体获得 +1 奖励, 其他智能体不共享该奖励。这一设定使得智能体之间缺乏共同目标, 协作行为 (如接力搬运) 难以自发涌现, 也是本报告所有分析的基本前提。

主要实验发现可归纳为三点。第一, SEAC 通过跨智能体经验共享实现了显著的样本效率提升, 收敛速度约为 IAC 的 N 倍, 与原论文一致。第二, SEAC 相较于 IAC 的训练 reward 优势在相当程度上来自熵不对称导致的采样偏差: SEAC 跨智能体损失项不含熵正则化, 使其策略天然低熵, 训练时的采样接近确定性推理, 从而获得虚高的 episodic reward。在统一温度对齐后, 各参数不共享算法之间未表现出显著的性能差距。第三, 在 SEAC 基础上引入纵向自模仿学习 (SEAC-SIL) 可以叠加经验复用的加速效果, 进一步缩短收敛时间。

未来工作可从以下几个方向展开。一是在全局奖励 (GLOBAL reward) 模式下重新评估各算法: 在全局奖励下, 所有智能体共享同一目标, 有可能自发形成接力搬运等有意义的协作行为, 而我也尝试了让 critic 额外接收所有智能体的位置信息 (ego-centered 联合观测), 但未观察到明显的性能提升——在 INDIVIDUAL 奖励下, 其他智能体的信息对自身价值估计帮助有限; 集中 Critic 在 GLOBAL 场景中或许才能真正发挥其价值函数信息量的优势。二是在策略网络中引入时序记忆 (如 RNN 或 LSTM), 使智能体能够借助历史信

息判断自身是否陷入死锁并主动跳出——当前纯前馈 MLP 在部分可观测的仓储环境中缺乏这一能力，而本文的温度扫描已表明引入适度不确定性有助于缓解死锁，但更好的方案是从网络结构层面提供时序感知。三是对碰撞行为施加显式的内在惩罚，以减少多智能体之间在狭窄通道中的步数浪费。这些方向受限于计算资源（tiny-2ag 训练 80M 步约 7 小时，small-4ag 训练 150M 步约需两天）的情况下未能进行验证。

回顾整个过程，我对 RL 算法中方差和偏差的实际作用有了更切身的体会。一个具体的例子是，当 n-step return 的步数从 5 提高到 20 甚至使用 MC 回报时，这类稀疏奖励任务的训练反而变得不稳定乃至无法收敛。原论文也指出 QMIX、MADDPG 和 ROMA 等方法在 RWARE 上未能收敛。这些经验说明，MARL 仍是一个对算法设计细节高度敏感的领域——步数长度、熵正则化、奖励结构、信用分配方式等每一个维度的调整都可能带来质变。对不同实验条件和评估方式的细致审视，或许比提出新的黑箱算法更为重要。

参考文献

Christianos, F., Schäfer, L., and Albrecht, S. V. Shared experience actor-critic for multi-agent reinforcement learning, 2021. URL <https://arxiv.org/abs/2006.07169>.